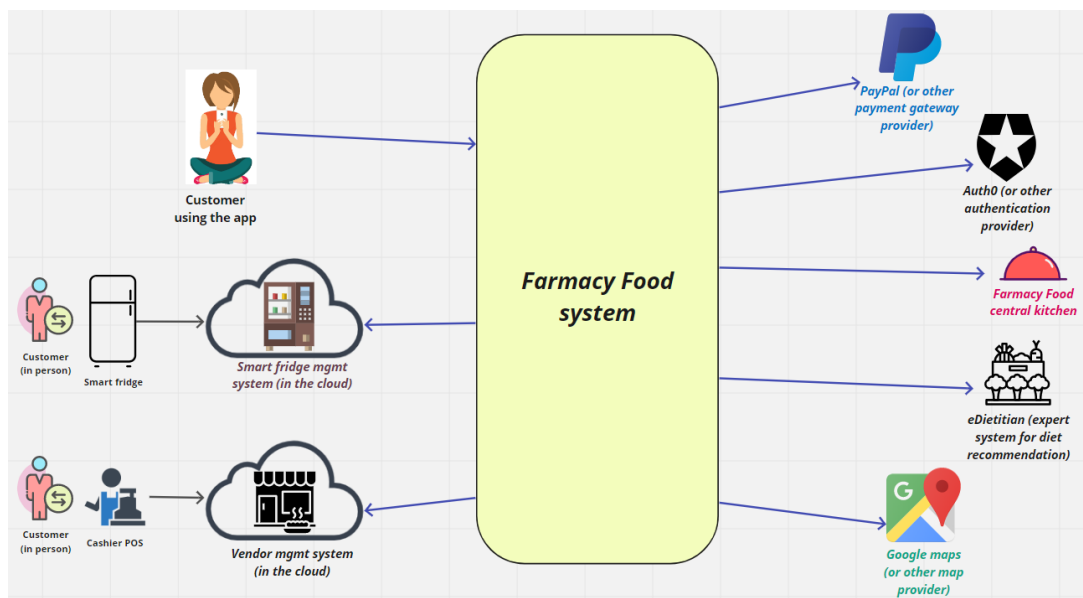


Trabajo Práctico Grupal: Farmacy Food

FarmacyFood es un startup que desarrolla productos relacionados con la comida saludable con una base tecnológica. La misión es dar respuesta a las necesidades dietarias de las personas mediante opciones de comidas sabrosas y que promuevan estilos de vida activos. Adicionalmente, se busca que estas comidas sean radicalmente accesibles y de bajo costo.

Esencialmente se busca instalar heladeras especiales (inteligentes) en distintos comercios, donde se distribuyan comida y viandas saludables en forma diaria, y que las personas puedan ver donde están y acceder a esta oferta desde sus celulares. De esta manera, las personas pueden dirigirse a estas heladeras, tomar la comida de su preferencia, y pagar en el lugar. Con el tiempo, se pretender personalizar la oferta a las preferencias y sugerencias de los usuarios.

La idea principal es la existencia de una “cocina fantasma” que prepara las distintas comidas y viandas, y a través de un sistema de software permite que los usuarios tengan visibilidad de los ítems disponibles, y puedan pasar a buscar sus ítems en los distintos puntos de venta. Un diagrama de contexto del sistema es el siguiente:



El trabajo debe centrarse en el backend, que orquesta todas las interacciones, si bien deben contemplarse en el diseño los sistemas adicionales (aplicaciones de usuarios, administrativas, de pagos, etc.)

FUNCIONALIDAD

1. Cada producto (vianda, snack, bebida) tiene: ID, nombre, descripción, categoría (vegetariana, vegana, sin gluten, etc.), precio, información nutricional (calorías, proteínas, etc.), atributos de temperatura de conservación.
2. Cada heladera tiene: ID, ubicación (latitud/longitud, dirección), capacidad máxima por tipo de producto, estado (operativa, en mantenimiento, fuera de servicio).
3. La heladera inteligente reporta cada vez que un producto es retirado (o devuelto). El sistema debe reflejar stock actualizado.
4. Un usuario selecciona productos de una heladera específica, genera una orden, paga, y el sistema autoriza la apertura de la heladera. Se debe registrar el detalle de la orden y el medio de pago.
5. La cocina fantasma necesita saber, para el día siguiente, qué productos y en qué cantidades deben prepararse, basado en ventas históricas, stock remanente y tendencias.
6. El sistema debe registrar las preferencias de los usuarios (tipos de dieta, alergias, productos favoritos) y sugerir productos en función de su historial (similar a "quienes compraron esto también compraron...").
7. Enviar alertas push cuando un producto favorito de un usuario vuelve a estar disponible en una heladera cercana.

CONSIDERACIONES DE ARQUITECTURA Y NO FUNCIONALES

- **Utilizar arquitectura de microservicios**
- **Comunicación:** Utilizar APIs REST, mayormente síncrono aunque podría ser asíncrono (fire-and-forget) si se requiere.
- **Persistencia:** Cada microservicio debe tener su propia base de datos (puede ser la misma tecnología, pero esquemas separados). Se valorará el uso de bases de datos diferentes (ej. PostgreSQL para transacciones, MongoDB para otros datos).
- **Resiliencia** – Implementar reintentos, timeouts, y fallback ante fallos de servicios externos (gateway de pagos, servicio de notificaciones).
- **Trazabilidad** – Cada operación crítica (compra, actualización de stock, producción) debe quedar registrada en un log centralizado.
- **Seguridad** – Autenticación y autorización con JWT (usuarios y roles: cliente, administrador de heladeras, cocina fantasma).

TAREAS DEL TRABAJO PRÁCTICO

- **Features y User Stories:**
 - Transformar los requerimientos dados en una serie de features y user stories.
- **Arquitectura de microservicios:**
 - Definir los microservicios, sus responsabilidades y API pública (contratos OpenAPI/Swagger).
 - Diagrama de módulos de la arquitectura de solución.
- **Modelo de datos:**
 - Diagrama entidad-relación por microservicio, indicando claves y relaciones entre servicios.
 - Estrategias de consistencia (eventual vs. fuerte), en caso que aplique

- **Decisiones tecnológicas:**
 - Justificar elección de las tecnologías.
 - Justificar uso de patrones: API Gateway, Circuit Breaker, BackForFrontend, etc.
- **Diseño de API REST:**
 - Listado de endpoints con método, ruta, parámetros, códigos de respuesta, ejemplos de request/response (JSON).
 - Documentar utilizando OpenAPI (Swagger), incluyendo descripciones, ejemplos y códigos de error.
- **Implementación:**
 - Desarrollar los microservicios necesarios en forma completa, y adicionalmente el soporte de JWT.
- **Testing:**
 - Incorporar pruebas unitarias y de integración para el backend.

Entregable: Repositorio Git con código fuente, archivo docker-compose.yml para levantar todos los servicios (bases de datos, gateway, etc.) y mínimamente README con instrucciones de ejecución y descripción del diseño.